

Lecture 6

Binary Search

15-122: Principles of Imperative Computation (Spring 2025)
Frank Pfenning

One of the fundamental and recurring problems in computer science is to find elements in collections, such as elements in sets. An important algorithm for this problem is *binary search*. We use binary search to look for an integer in a sorted array to exemplify it. We started in a previous lecture by discussing *linear search* and giving some background on the problem. This lecture clearly illustrates the power of *order* in algorithm design: if an array is sorted we can search through it very efficiently, much more efficiently than when it is not ordered.

We will also once again see the importance of loop invariants in writing correct code. Here is a note by Jon Bentley about binary search:

I've assigned [binary search] in courses at Bell Labs and IBM. Professional programmers had a couple of hours to convert [its] description into a program in the language of their choice; a high-level pseudo-code was fine. At the end of the specified time, almost all the programmers reported that they had correct code for the task. We would then take thirty minutes to examine their code, which the programmers did with test cases. In several classes and with over a hundred programmers, the results varied little: ninety percent of the programmers found bugs in their programs (and I wasn't always convinced of the correctness of the code in which no bugs were found).

*I was amazed: given ample time, only about ten percent of professional programmers were able to get this small program right. But they aren't the only ones to find this task difficult: in the history in Section 6.2.1 of his *Sorting and Searching*, Knuth points out that while the first binary search was published in 1946, the first published binary search without bugs did not appear until 1962.*

—Jon Bentley, *Programming Pearls* (1st edition), pp.35–36

I contend that what these programmers are missing is the understanding of how to use loop invariants in composing their programs. They help

us to make assumptions explicit and clarify the reasons *why* a particular program is correct. Part of the magic of pre- and post-conditions as well as loop invariants and assertions is that they *localize* reasoning. Rather than having to look at the whole program, or the whole function, we can focus on individual statements tracking properties via the loop invariants and assertions.

Additional Resources

- [Review slides](https://cs.cmu.edu/~15122/handouts/slides/review/06-binsearch.pdf) (https://cs.cmu.edu/~15122/handouts/slides/review/06-binsearch.pdf)
- [Code for this lecture](https://cs.cmu.edu/~15122/handouts/code/06-binsearch.tgz) (https://cs.cmu.edu/~15122/handouts/code/06-binsearch.tgz)

The learning goals for this lecture are as follows:

Computational Thinking: Obtaining an exponential speed-up by partitioning the problem space — a prelude to a more general technique called divide-and-conquer.

Algorithms and Data Structures: Binary search.

Programming: Using loop invariants as a design tool for programs.

1 Binary Search

Can we do better than searching through the array linearly? If you don't know the answer already it might be surprising that, yes, we can do *significantly* better! Perhaps almost equally surprising is that the code is almost as short! However, this will require the array to be *sorted*.

Before we write the code, let us describe the algorithm. We start searching for x by examining the *middle element* of the sorted array. If it is smaller than x , then x must be in the upper half of the array (if it is there at all); if it is greater than x , then x must be in the lower half. Now we continue by restricting our attention to either the upper or lower half, again finding the middle element and proceeding as before.

We stop if we either find x , or if the size of the subarray shrinks to zero, in which case x cannot be in the array.

Before we write a program to implement this algorithm, let us analyze the running time. Assume for the moment that the size of the array is a power of 2, say 2^k . Each time around the loop, when we examine the middle element, we cut the size of the subarrays we look at in half. So before the first iteration the size of the subarray of interest is 2^k . After the first iteration (i.e., just before the second), it is of size 2^{k-1} , then 2^{k-2} , etc. After

k iterations it will be $2^{k-k} = 1$, so we stop after the next iteration. Altogether we can have at most $k + 1$ iterations. Within each iteration, we perform a constant amount of work: computing the midpoint, and a few comparisons. So, overall, when given a size of array n we perform $c \times \log_2 n$ operations (for some constant c).¹

If the size n is not a power of 2, then we can round n up to the next power of 2, and the reasoning above still applies. For example, if $n = 13$ we round it up to $16 = 2^4$. The actual number of steps can only be smaller than this bound, because some of the actual subintervals may be smaller than the bound we obtained when rounding up n .

The logarithm grows much more slowly than the linear function that we obtained when analyzing linear search. As before, suppose we double the size of the input, $n' = 2 \times n$. Then the number of operations will be $c \times \log(2 \times n) = c \times (\log 2 + \log n) = c \times (1 + \log n) = c + c \times \log n$. So the number of operations increases only by a constant amount c when we double the size of the input. Considering that the largest representable positive number in 32-bit two's complement representation is $2^{31} - 1$ (about 2 billion) binary search even for unreasonably large arrays will only traverse the loop 31 times!

2 Implementing Binary Search

The specification for binary search is the same as for linear search.

```

1 int binsearch(int x, int[] A, int n)
2 //@requires 0 <= n && n <= \length(A);
3 //@requires is_sorted(A, 0, n);
4 /*@ensures (-1 == \result && !is_in(x, A, 0, n))
5           || ((0 <= \result && \result < n) && A[\result] == x);
6 @*/
7 ;

```

We declare two variables, `lo` and `hi`, which hold the lower and upper end of the subinterval in the array that we are considering. We start with `lo` as 0 and `hi` as n , so the interval includes `lo` and excludes `hi`. This often turns out to be a convenient choice when computing with arrays (but see Exercise 1).

The **for** loop from linear search becomes a **while** loop, exiting when the interval has size zero, that is, `lo == hi`. We can easily write the first

¹In general in computer science, we are mostly interested in logarithm to the base 2 so we will just write $\log n$ for log to the base 2 from now on unless we are considering a different base.

loop invariant, relating *lo* and *hi* to each other and the overall bound of the array.

```

1 int binsearch(int x, int[] A, int n)
2 //@requires 0 <= n && n <= \length(A);
3 //@requires is_sorted(A, 0, n);
4 /*@ensures (-1 == \result && !is_in(x, A, 0, n))
5           || ((0 <= \result && \result < n) && A[\result] == x);
6 @*/
7 {
8     int lo = 0;
9     int hi = n;
10    while (lo < hi)
11        //@loop_invariant 0 <= lo && lo <= hi && hi <= n;
12    {
13        // ...??...
14    }
15    return -1;
16 }
```

In the body of the loop, we first compute the midpoint *mid*. By elementary arithmetic it is indeed between *lo* and *hi*.

Next in the loop body we check if $A[mid] = x$. If so, we have found the element and return *mid*.

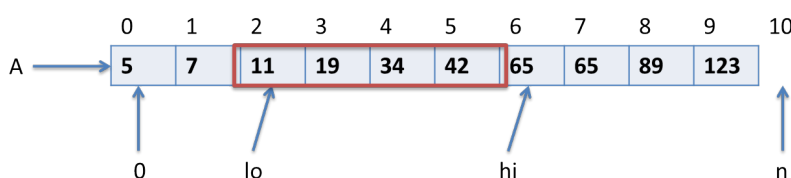
```

1 int binsearch(int x, int[] A, int n)
2 //@requires 0 <= n && n <= \length(A);
3 //@requires is_sorted(A, 0, n);
4 /*@ensures (-1 == \result && !is_in(x, A, 0, n))
5           || ((0 <= \result && \result < n) && A[\result] == x);
6 @*/
7 {
8     int lo = 0;
9     int hi = n;
10    while (lo < hi)
11        //@loop_invariant 0 <= lo && lo <= hi && hi <= n;
12        //@loop_invariant ...??...;
13    {
14        int mid = lo + (hi-lo)/2;
15        //@assert lo <= mid && mid < hi;
16        if (A[mid] == x) return mid;
17        // ...??...
18    }
19    return -1;
```

20 }

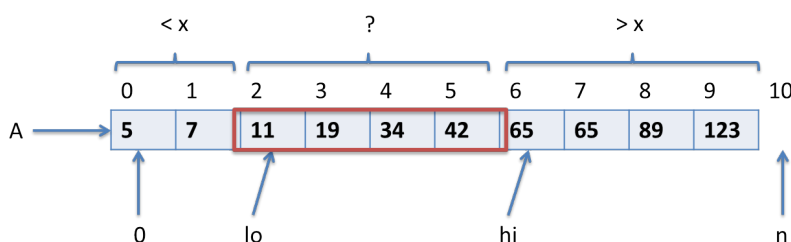
Now comes the hard part. What is the missing part of the invariant? The first instinct might be to say that x should be in the interval from $A[lo]$ to $A[hi]$. But that may not even be true when the loop is entered the first time.

Let's consider a generic situation in the form of a picture and collect some ideas about what might be appropriate loop invariants. Drawing diagrams to reason about an algorithm and the code that we are trying to construct is an extremely helpful general technique.



The red box around elements 2 through 5 marks the segment of the array still under consideration. This means we have *ruled out* everything to the right of (and including) hi and to the left of (and not including) lo . Everything to the left is ruled out, because those values have been recognized to be strictly less than x , while the ones on the right are known to be strictly greater than x , while the middle is still unexplored.

We can depict this as follows:



We can summarize this by stating that $A[lo - 1] < x$ and $A[hi] > x$. This implies that x cannot be in the segments $A[0..lo)$ and $A[hi..n)$ because the array is sorted (so all array elements to the left of $A[lo - 1]$ will also be less than x and all array elements to the right of $A[hi]$ will also be greater than x). For an alternative, see Exercise 2.

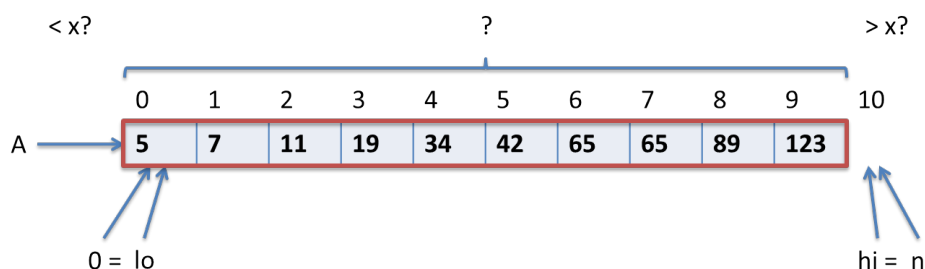
We can postulate these as invariants in the code.

```
1 int binsearch(int x, int[] A, int n)
2 //@requires 0 <= n && n <= \length(A);
3 //@requires is_sorted(A, 0, n);
4 /*@ensures (-1 == \result && !is_in(x, A, 0, n))
5           || ((0 <= \result && \result < n) && A[\result] == x);
6   @*/
7 {
8     int lo = 0;
9     int hi = n;
10    while (lo < hi)
11        //@loop_invariant 0 <= lo && lo <= hi && hi <= n;
12        //@loop_invariant A[lo-1] < x;
13        //@loop_invariant A[hi] > x;
14        {
15            int mid = lo + (hi-lo)/2;
16            if (A[mid] == x) return mid;
17            // ...??...
18        }
19    return -1;
20 }
```

Now a very powerful programming instinct should tell you something is fishy. Can you spot the problem with the new invariants even before writing any more code in the body of the loop?

Whenever you access an element of an array, you must have good reason to know that the access will be in bounds!

In the code we blithely wrote $A[lo - 1]$ and $A[hi]$ because they were in the middle of the array in our diagram. But initially (and potentially through many iterations) this may not be the case. Fortunately, it is easy to fix, following what we did for linear search. Consider the following picture when we start the search.



In this case all elements of the array have to be considered candidates. All elements strictly to the left of 0 (of which there are none) and to the right of n (of which there are none) have been ruled out. As in linear search, we can add this to our invariant using disjunction.

```

1 int binsearch(int x, int[] A, int n)
2 //@requires 0 <= n && n <= \length(A);
3 //@requires is_sorted(A, 0, n);
4 /*@ensures (-1 == \result && !is_in(x, A, 0, n))
5           || ((0 <= \result && \result < n) && A[\result] == x);
6 @*/
7 {
8     int lo = 0;
9     int hi = n;
10    while (lo < hi)
11        //@loop_invariant 0 <= lo && lo <= hi && hi <= n;
12        //@loop_invariant (lo == 0 || A[lo-1] < x);
13        //@loop_invariant (hi == n || A[hi] > x);
14        {
15            int mid = lo + (hi-lo)/2;
16            if (A[mid] == x) return mid;
17            // ...??...
18        }
19    return -1;
20 }

```

At this point, let's check if the loop invariant is strong enough to imply the post-condition of the function. If we return from inside the loop because $A[mid] = x$ we return mid , so $A[\text{return}] == x$ as required.

If we exit the loop because $lo < hi$ is false, we know $lo = hi$, by the first loop invariant. Now we have to distinguish some cases.

1. If $A[lo - 1] < x$ and $x < A[hi]$, then $x < A[lo]$ (since $lo = hi$). Because the array is sorted, x cannot be in it.
2. If $lo = 0$, then $hi = 0$. By the third loop invariant, then either $n = 0$ (and so the array has no elements and we must return -1), or $A[hi] = A[lo] = A[0] > x$. Because A is sorted, x cannot be in A if its first element is already strictly greater than x .
3. If $hi = n$, then $lo = n$. By the second loop invariant, then either $n = 0$ (and so we must return -1), or $A[n - 1] = A[hi - 1] = A[lo - 1] < x$. Because A is sorted, x cannot be in A if its last element is already strictly less than x .

Notice that we could verify all this without even knowing the complete program! As long as we can finish the loop to preserve the invariant and terminate, we will have a correct implementation! This would again be a good point for you to interrupt your reading and to try to complete the loop, reasoning from the invariant.

We have already tested if $A[mid] = x$. If not, then $A[mid]$ must be less or greater than x . If it is less, then we can keep the upper end of the interval as is, and set the lower end to $mid + 1$. Now $A[lo - 1] < x$ (because $A[mid] < x$ and $lo = mid + 1$), and the condition on the upper end remains unchanged.

If $A[mid] > x$ we can set hi to mid and keep lo the same. We do not need to test this last condition, because the fact that the tests $A[mid] = x$ and $A[mid] < x$ both failed implies that $A[mid] > x$. We note this in an assertion.


```
1 int binsearch(int x, int[] A, int n)
2 //@requires 0 <= n && n <= \length(A);
3 //@requires is_sorted(A, 0, n);
4 /*@ensures (-1 == \result && !is_in(x, A, 0, n))
5           || ((0 <= \result && \result < n) && A[\result] == x);
6   @*/
7 { int lo = 0;
8   int hi = n;
9   while (lo < hi)
10     //@loop_invariant 0 <= lo && lo <= hi && hi <= n;
11     //@loop_invariant (lo == 0 || A[lo-1] < x);
12     //@loop_invariant (hi == n || A[hi] > x);
13     {
14       int mid = lo + (hi-lo)/2;
15       //@assert lo <= mid && mid < hi;
16       if (A[mid] == x) return mid;
17       else if (A[mid] < x) lo = mid+1;
18       else /*@assert(A[mid] > x);@*/
19         hi = mid;
20     }
21   return -1;
22 }
```

Let's set up the proof of the loop invariants more schematically.

Init: When the loop is first reached, we have $lo = 0$ and $hi = n$, so the first loop invariant follows from the precondition to the function. Furthermore, the first disjunct in loop invariants two ($lo == 0$) and three ($hi == n$) is satisfied.

Preservation: Assume the loop invariants are satisfied and we enter the loop:

$$\begin{array}{ll} 0 \leq lo \leq hi \leq n & (\text{Inv 1}) \\ (lo = 0 \text{ or } A[lo - 1] < x) & (\text{Inv 2}) \\ (hi = n \text{ or } A[hi] > x) & (\text{Inv 3}) \\ lo < hi & (\text{loop condition}) \end{array}$$

We compute $mid = lo + \lfloor (hi - lo) / 2 \rfloor$. Now we distinguish three cases:

$A[mid] = x$: In that case we exit the function, so we don't need to show preservation. We do have to show the post-condition, but we already considered this earlier in the lecture.

$A[mid] < x$: Then

$$\begin{array}{l} lo' = mid + 1 \\ hi' = hi \end{array}$$

The first loop invariant $0 \leq lo' \leq hi' \leq n$ follows from the formula for mid , our assumptions, and elementary arithmetic.

For the second loop invariant, we calculate:

$$\begin{array}{ll} A[lo' - 1] &= A[(mid + 1) - 1] \quad \text{since } lo' = mid + 1 \\ &= A[mid] \quad \text{by arithmetic} \\ &< x \quad \text{this case } A[mid] < x \end{array}$$

The third loop invariant is preserved, since $hi' = hi$.

$A[mid] > x$: Then

$$\begin{array}{l} lo' = lo \\ hi' = mid \end{array}$$

Again, by elementary arithmetic, $0 \leq lo' \leq hi' \leq n$.

The second loop invariant is preserved since $lo' = lo$.

For the third loop invariant, we calculate

$$\begin{array}{ll} A[hi'] &= A[mid] \quad \text{since } hi' = mid \\ &> x \quad \text{since we are in the case } A[mid] > x \end{array}$$

3 Termination

Does this function terminate? If the loop body executes, that is, $lo < hi$, then the interval from lo to hi is non-empty. Moreover, the intervals from lo to mid and from $mid + 1$ to hi are both strictly smaller than the original interval. Unless we find the element, the difference between hi and lo must eventually become 0 and we exit the loop.

4 One More Observation

You might be tempted to calculate the midpoint with

```
13 int mid = (lo + hi)/2;
```

but that is in fact incorrect. Consider this change and try to find out why this would introduce a bug.

Were you able to see it? It's subtle, but somewhat related to other problems we had. When we compute $(lo + hi)/2$; we could actually have an overflow, if $lo + hi > 2^{31} - 1$. This is somewhat unlikely in practice, since $2^{31} = 2G$, about 2 billion, so the array would have to have at least 1 billion elements. This is not impossible, and, in fact, a bug like this in the Java libraries² was actually exposed.

Fortunately, the fix is simple: because $lo < hi$, we know that $hi - lo > 0$ and represents the size of the interval. So we can divide that in half and add it to the lower end of the interval to get its midpoint.

```
13 int mid = lo + (hi-lo)/2;    // as shown in binary search
14 //@assert lo <= mid && mid < hi;
```

Let us convince ourselves why the assert is correct. The division by two will round to zero, *down* to 0 here, because $hi - lo > 0$. Thus, $0 \leq (hi - lo)/2 < hi - lo$, because dividing a positive number by two will make it strictly smaller. Hence,

$$mid = lo + (hi - lo)/2 < lo + (hi - lo) = hi$$

Since dividing positive numbers by two will still result in a non-negative number, the first part of the assert is correct as well.

$$mid = lo + (hi - lo)/2 \geq lo + 0 = lo$$

Other operations in this binary search take place on quantities bounded from above by the **int** n and thus cannot overflow.

Why did we choose to look at the middle element and not another element at all? Because, whatever the outcome of our comparison to that middle element may be, we maximize how much we have learned about the contents of the array by doing this one comparison. If we find the element, we are happy because we are done. If the middle element is smaller than what we are looking for, however, we are happy as well, because we have just learned that the lower half of the array has become irrelevant. Similarly, if the middle element is bigger, then we have made substantial progress by learning that we never need to look at the upper half of the array anymore. There are other choices, however, where binary search will also still work in essentially the same way.

5 Some Measurements

Algorithm design is an interesting mix of mathematics and an experimental science. Our analysis above, albeit somewhat preliminary in nature, allow

²See Joshua Bloch's [Extra](#), [Extra](#) blog entry.

us to make some predictions of running times of our implementations. We start with linear search. We first set up a file to do some experiments. We assume we have already tested our functions for correctness, so only timing is at stake. See the file `search-time.c0` in the code directory for this lecture. We compile this file, together with our implementation from this lecture, with the `cc0` command below. We can get an overall end-to-end timing with the Unix `time` command. Note that we do not use the `-d` flag, since that would dynamically check contracts and completely throw off our timings.

```
% cc0 find.c0 find-time.c0
% time ./a.out
```

When running linear search 2000 times (1000 times with x in the array, and 1000 times with random x) on 2^{18} elements (256 K elements) we get the following answer

```
Timing 1000 times with 2^18 elements
0
4.602u 0.015s 0:04.63 99.5% 0+0k 0+0io 0pf+0w
```

which indicates 4.602 seconds of user time.

Running linear search 2000 times on random arrays of size 2^{18} , 2^{19} and 2^{20} we get the timings on our MacBook Pro

array size	time (secs)
2^{18}	4.602
2^{19}	9.027
2^{20}	19.239

The running times are fairly close to doubling consistently. Due to memory locality effects and other overheads, for larger arrays we would expect larger numbers.

Running the same experiments with binary search we get

array size	time (secs)
2^{18}	0.020
2^{19}	0.039
2^{20}	0.077

which is much, much faster but looks suspiciously linear as well.

Reconsidering the code we see that the time might increase linearly because we actually must iterate over the whole array in order to initialize it with random elements!

We comment out the testing code to measure only the initialization time, and we see that for 2^{20} elements we measure 0.072 seconds, as compared to 0.077 which is insignificant. Effectively, we have been measuring the time to set up the random array, rather than to find elements in it with binary search!

This is a vivid illustration of the power of divide-and-conquer. Logarithmic running time for algorithms grow very slowly, a crucial difference to linear-time algorithms when the data sizes become large.

6 Exercises

Exercise 1 (sample solution on page 17). Rewrite the body of the binary search function so that both lower and upper bounds of the interval examined in the loop are inclusive. Make sure to update the loop invariants appropriately. Finally, prove the validity of the updated loop invariants.

Exercise 2 (sample solution on page 20). Rewrite the invariants of the binary search function to use `is_in(x, A, lo, hi)` which returns `true` if and only if there is an `i` such that `x == A[i]` for `lo <= i < u`. The function `is_in(x, A, lo, hi)` assumes that `0 <= lo <= hi <= \length(A)`.

Then prove the new loop invariants, and verify that they are strong enough to imply the function's post-condition. As you do so you may assume obvious properties of `is_in`.

Exercise 3. Binary search as presented here may not find the leftmost occurrence of `x` in the array in case the occurrences are not unique. Given an example demonstrating this.

Now change the binary search function and its loop invariants so that it will always find the leftmost occurrence of `x` in the given array (if it is actually in the array, `-1` as before if it is not).

Prove the loop invariants and the post-conditions for this new version, and verify termination.

Exercise 4 (sample solution on page 23). If you were to replace the midpoint computation by

```
int mid = (lo + hi)/2;
```

then which part of the contract will alert you to a flaw in your thinking? Why? Give an example showing how the contracts can fail in that case.

Exercise 5 (sample solution on page 23). Because computing the midpoint of an array segment as $(lo + hi)/2$ can cause an overflow, it is tempting to define it instead as

```
int mid = lo/2 + hi/2;
```

Show that, in regular integer arithmetic (where overflows never occur), the two expressions $\frac{lo}{2} + \frac{hi}{2}$ and $\frac{lo+hi}{2}$ do not always produce the same result (note that we are asking about in integer arithmetic, where for example $\frac{1}{2} = 0$).

Exercise 6 (sample solution on page 23). Because computing the midpoint of an array segment as $(lo + hi)/2$ can cause an overflow, we defined it instead as

```
int mid = lo + (hi - lo)/2;
```

Show that, in regular integer arithmetic (where overflows never occur), the two expression $lo + \frac{hi-lo}{2}$ and $\frac{lo+hi}{2}$ always produce the same result (note that we are asking about in integer arithmetic, where for example $\frac{1}{2} = 0$).

Exercise 7 (sample solution on page 25). In lecture, we used design-by-invariant to construct the loop body implementation from the loop invariant that we have identified before. We could also have maintained the loop invariant by replacing the whole loop body just with

```
// .... loop_invariant elided ....  
{  
    lo = lo;  
    hi = hi;  
}
```

Prove the loop invariants for this loop body. What is wrong with this choice? Which part of our proofs fail, thereby indicating why this loop body would not implement binary search correctly?

Sample Solutions

Solution of exercise 1 For the upper bound to be inclusive, we need to initialize the variable `hi` to `n-1`. Most of our original code for binary search remains unchanged, but a few parts need to be modified. Let's see the resulting code:

```

1 int binsearch_inclusive(int x, int[] A, int n)
2 //@requires n == \length(A);
3 //@requires is_sorted(A, 0, n);
4 /*@ensures (-1 == \result && !is_in(x, A, 0, n))
5           || ((0 <= \result && \result < n) && A[\result] == x); @*/
6 {
7     int lo = 0;
8     int hi = n-1;
9
10    while (lo <= hi)
11        //@loop_invariant 0 <= lo && lo <= hi+1 && hi < n;
12        //@loop_invariant gt_seg(x, A, 0, lo);
13        //@loop_invariant lt_seg(x, A, hi+1, n);
14    {
15        int mid = lo + (hi-lo)/2;
16        //@assert lo <= mid && mid <= hi;
17        if (A[mid] == x) return mid;
18        else if (A[mid] < x)
19            lo = mid+1;
20        else /*@assert(A[mid] > x);@*/
21            hi = mid-1;
22    }
23    //@assert lo == hi+1;
24
25    return -1;
26 }
```

The only change to the code within the loop is to update `hi` to `mid-1` instead of `mid` and the loop guard to `lo <= hi`. However, we need to be careful with the contracts. Now that the loop guard allows `lo == hi`, we need `lo <= hi+1` as our loop invariant. (Also consider what happens when calling `binsearch_inclusive` on the empty array: then `n == 0`, which means that `hi == -1`.) Other changes are more immediate: since we set `hi` to `n-1`, a sufficient loop invariant relating these two variables is `hi < n`. Our assertion about `mid` now must allow `mid == hi`, again because the loop guard

allows $lo == hi$. Finally, the call to `lt_seg` of our original code is adapted by simply using $hi+1$ in place of hi as the third argument, since hi itself is now unexplored.

Our original reasoning about the assertion on line 16 still holds, except that $hi-lo$ may no longer be strictly positive, but instead only non-negative. This is accounted for by allowing $mid == hi$.

We now prove that the updated loop invariants are valid, i.e., that they hold initially and are preserved by an arbitrary iteration of the loop. Let's start with $lo \leq hi+1 \ \&\& \ hi < n$.

INIT We need to show that the loop invariant on line 11 is true initially. The first conjunct is unchanged, so we will focus on the other two conjuncts.

To Show: $lo \leq hi+1 \ \&\& \ hi < n$ is true initially

- a. $n == \text{length}(A)$ by line 2
- b. $n \geq 0$ by postcondition of `length`
- c. $hi == n-1$ by line 8
- d. $hi < n$ by math on (b) and (c)
- e. $lo == 0$ by line 7
- f. $lo \leq hi+1$ by math on (b), (c) and (e)

PRES We need to show that the loop invariant on line 11 is preserved by an arbitrary iteration of the loop. Again, we focus on the other two conjuncts.

Assume: $lo \leq hi+1 \ \&\& \ hi < n$

To Show: $lo' \leq hi'+1 \ \&\& \ hi' < n$

We shall consider the cases where $A[mid] < x$ and $A[mid] > x$. We ignore the case where $A[mid] == x$ as the function returns immediately and so there is no preservation to prove.

Case $A[mid] < x$:

- a. $lo' = mid+1$ by case assumption, lines 18 and 19
- b. $hi' = hi$ by (case assumption, unchanged)
- c. $hi' < n$ by math on assumption and (b)
- d. $mid \leq hi$ by line 16
- e. $mid+1 \leq hi+1$ by math on (d)
- f. $lo' \leq hi'+1$ by (a), (b) and (e)

Case $A[mid] > x$:

- a. $lo' = lo$ by (case assumption, unchanged)

<i>b.</i> $hi' = mid - 1$	by case assumption, lines 20 and 21
<i>c.</i> $mid < n$	by line 16 and assumption
<i>d.</i> $hi' < n$	by (b) and (c)
<i>e.</i> $lo' \leq mid$	by line 16, (a)
<i>f.</i> $lo' \leq hi' + 1$	by math on (b) and (e)

The proof that $gt_seg(x, A, 0, lo)$ is a valid loop invariant remains unchanged, because the way we initialize and update lo is unchanged.

INIT We need to show that the loop invariant on line 13 is true initially.

To Show: $x < A[hi+1, n]$ is true initially

<i>a.</i> $x < A[n, n]$	by definition of lt_seg (empty segment)
<i>b.</i> $hi == n - 1$	by line 8
<i>c.</i> $hi + 1 == n$	by math on (b)
<i>d.</i> $x < A[hi + 1, n]$	by math on (a) and (c)

PRES We need to show that the loop invariant on line 13 is preserved by an arbitrary iteration of the loop.

Assume: $x < A[hi + 1, n]$

To Show: $x < A[hi' + 1, n]$

The only situation where hi changes is when $A[mid] > x$. We can limit ourselves to proving just this case.

<i>a.</i> $A[0, n]$ sorted	by line 3
<i>b.</i> $0 \leq mid$	by math on lines 11 and 16
<i>c.</i> $mid < n$	by math on lines 11 and 16
<i>d.</i> $x < A[mid]$	by case assumption
<i>e.</i> $x < A[mid, n]$	by math on (a), (b), (c), and (d)
<i>f.</i> $hi' = mid - 1$	by lines 20 and 21
<i>g.</i> $x < A[hi' + 1, n]$	by math on (e) and (f)

Finally, to prove the postcondition of this updated function, we can limit ourselves to the case where we return on line 25, because the internal return is mostly unchanged (the change in the assertion about mid is compensated for by the change in the first loop invariant).

EXIT To Show: $!is_in(x, A, 0, n)$ when returning on line 25

<i>a.</i> $lo > hi$	by line 10
<i>b.</i> $lo \leq hi + 1$	by line 11

- c.* $lo == hi+1$ by math on (a) and (b)
- d.* $x > A[0, lo)$ by line 12
- e.* $x > A[0, hi+1)$ by math on (d) and (c)
- f.* $x < A[hi+1, n)$ by line 13
- g.* $!is_in(x, A, 0, n)$ by math on (e) and (f)

Termination is similar to before, except that $hi - lo$ is now bounded by -1 instead of 0 .

Solution of exercise 2

To implement this variant of binary search, we simply update the code seen in this lecture to use $!is_in(x, A, 0, lo)$ and $!is_in(x, A, hi, n)$ as the loop invariants concerning lo and hi , respectively. Here's the updated code, where the included key reasoning steps (see below) as assertions.

```

1 int search(int x, int[] A, int n)
2 //@requires n == \length(A);
3 //@requires is_sorted(A, 0, n);
4 /*@ensures (\result == -1 && !is_in(x, A, 0, n))
5           || (0 <= \result && \result < n && A[\result] == x); @*/
6 {
7     int lo = 0;
8     int hi = n;
9
10    while (lo < hi)
11        //@loop_invariant 0 <= lo && lo <= hi && hi <= n;
12        //@loop_invariant !is_in(x, A, 0, lo);
13        //@loop_invariant !is_in(x, A, hi, n);
14    {
15        int mid = lo + (hi - lo)/2;
16        //@assert lo <= mid && mid < hi;
17
18        if (A[mid] == x) return mid;
19        if (A[mid] < x) {
20            //@assert mid + 1 <= hi;
21            //@assert !is_in(x, A, 0, mid+1);
22            lo = mid+1;
23        } else { //@assert A[mid] > x;
24            //@assert !is_in(x, A, mid, n);
25            hi = mid;
26        }
27    }
28    //@assert lo == hi;
29    //@assert !is_in(x, A, 0, n);
30    return -1;
31 }

```

Let's prove that the new loop invariants support the correctness of this code. We will start with the EXIT assuming that these loop invariants are valid.

The function exits in two place, on lines 18 and 30. The first of this returns is proved exactly as for the code seen in this lecture, in particular it hits the second disjunct of the postcondition (on line 5). We will instead focus on the second return statement, on line 30, for which we need to prove the first disjunct (on line 4), and specifically that `!is_in(x, A, 0, n)`.

To Show: `!is_in(x, A, 0, n)`

a. `lo == hi` by lines 10 and 11

- b. $!is_in(x, A, 0, lo)$ by line 12
- c. $!is_in(x, A, hi, n)$ by line 13
- d. $!is_in(x, A, 0, n)$ by math on (a), (b) and (c)

Next, let's prove that the new loop invariants (on lines 12 and 13) are valid. We start with the loop invariant on line 12.

INIT We need to show that the loop invariant on line 12 is true initially.

To Show: $!is_in(x, A, 0, lo)$ is true initially

- a. $!is_in(x, A, 0, 0)$ by definition of is_in
- b. $lo == 0$ by line 7
- c. $!is_in(x, A, 0, lo)$ by math on (a) and (b)

PRES We need to show that the loop invariant on line 12 is preserved by an arbitrary iteration of the loop.

Assume: $!is_in(x, A, 0, lo)$

To Show: $!is_in(x, A, 0, lo')$

The only situation where lo changes is when $A[mid] < x$. We can limit ourselves to proving just this case.

- a. $A[0, n)$ sorted by line 3
- b. $!is_in(x, A, 0, lo)$ by assumption
- c. $0 \leq mid+1$ by math on lines 11 and 16
- d. $mid+1 \leq n$ by math on line 16
- e. $x < A[mid]$ by line 19
- f. $!is_in(x, A, 0, mid+1)$ by math on (a), (b), (c), (d) and (e)
- g. $lo' = mid+1$ by lines 19 and 22
- h. $!is_in(x, A, 0, lo')$ by math on (g)

The proof that the loop invariant on line 13 is valid is similar.

INIT We need to show that the loop invariant on line 13 is true initially.

To Show: $!is_in(x, A, hi, n)$ is true initially

- a. $!is_in(x, A, n, n)$ by definition of is_in
- b. $hi == n$ by line 8
- c. $!is_in(x, A, hi, n)$ by math on (a) and (b)

PRES We need to show that the loop invariant on line 13 is preserved by an arbitrary iteration of the loop.

Assume: `!is_in(x, A, hi, n)`

To Show: `!is_in(x, A, hi', n)`

The only situation where `hi` changes is when `A[mid] > x`. We can limit ourselves to proving just this case.

- | | |
|--------------------------------------|---------------------------------------|
| a. <code>A[0, n)</code> sorted | by line 3 |
| b. <code>!is_in(x, A, hi, n)</code> | by assumption |
| c. <code>mid < hi</code> | by line 16 |
| d. <code>hi <= n</code> | by line 11 |
| e. <code>x > A[mid]</code> | by line 23 |
| f. <code>!is_in(x, A, mid, n)</code> | by math on (a), (b), (c), (d) and (e) |
| g. <code>hi' = mid</code> | by lines 23 and 25 |
| h. <code>!is_in(x, A, hi', n)</code> | by math on (g) |

Solution of exercise 4 Computing `mid` as $(lo + hi)/2$ will result in a negative value whenever `lo + hi` overflows. For example, if `lo == 1` and `hi == int_max()`, then `lo + hi == int_min()` and therefore `lo + hi` is equal to `int_min()` (i.e., -1073741824).

Most immediately, this would be caught by the assertion `lo <= mid && mid < hi` right after the calculation of `mid`.

Were we to remove this assertion, the issue would manifest when evaluating `A[mid]` since the index `mid` is negative. The program would abort trying to evaluate this expression.

Solution of exercise 5 These two expressions will produce different result whenever both `lo` and `hi` are odd. Assume for example that both are equal to 1.

Then

$$\frac{lo}{2} + \frac{hi}{2} = \frac{1}{2} + \frac{1}{2} = 0 + 0 = 0$$

But

$$\frac{lo + hi}{2} = \frac{1 + 1}{2} = \frac{2}{2} = 1$$

Solution of exercise 6 Just like in binary search, we shall assume that $0 \leq lo \leq hi$, and therefore that we are dealing exclusively with non-negative quantities.

First a reminder about regular integer arithmetic:

- If a is an even number $2b$, then $\frac{a}{2} = b = \frac{a+1}{2}$.
- If a is an odd number $2b + 1$, then $\frac{a}{2} = b = \frac{a-1}{2}$.

To prove that $lo + \frac{hi-lo}{2}$ is equal to $\frac{lo+hi}{2}$ in integer arithmetic, we need to distinguish cases on whether lo and hi are even or odd. That's a total of four cases.

lo and hi are both even Then, $lo = 2m$ for some m and $hi = 2n$ for some n .

Observe that, in this case, $\frac{lo+hi}{2} = m + n$.

Then,

$$\begin{aligned}
 lo + \frac{hi-lo}{2} &= 2m + \frac{2n-2m}{2} \\
 &= 2m + (n-m)\frac{2}{2} \\
 &= 2m + (n-m) \\
 &= m + n \\
 &= \frac{2m+2n}{2} \\
 &= \frac{lo+hi}{2}
 \end{aligned}$$

lo is even and hi is odd Then, $lo = 2m$ for some m and $hi = 2n + 1$ for some n .

Observe that, in this case, $\frac{lo+hi}{2} = m + n$.

Then,

$$\begin{aligned}
 lo + \frac{hi-lo}{2} &= 2m + \frac{2n+1-2m}{2} \\
 &= 2m + (n-m)\frac{2}{2} + \frac{1}{2} \\
 &= 2m + (n-m) + 0 \\
 &= m + n \\
 &= \frac{2m}{2} + \frac{2n+1}{2} \\
 &= \frac{2m+2n+1}{2} \\
 &= \frac{lo+hi}{2}
 \end{aligned}$$

lo is odd and hi is even Then, $lo = 2m + 1$ for some m and $hi = 2n$ for some n .

Observe that, in this case, $\frac{lo+hi}{2} = m + n$.

Then,

$$\begin{aligned}
 lo + \frac{hi-lo}{2} &= 2m + 1 + \frac{2n-(2m+1)}{2} \\
 &= 2m + 1 + \frac{2n-(2m+2)+1}{2} \\
 &= 2m + 1 + (n - m - 1)\frac{2}{2} + \frac{1}{2} \\
 &= 2m + 1 + (n - m - 1) + 0 \\
 &= m + n \\
 &= \frac{2m+1}{2} + \frac{2n}{2} \\
 &= \frac{2m+1+2n}{2} \\
 &= \frac{lo+hi}{2}
 \end{aligned}$$

lo and hi are both odd Then, $lo = 2m + 1$ for some m and $hi = 2n + 1$ for some n .

Observe that, in this case, $\frac{lo+hi}{2} = m + n + 1$.

Then,

$$\begin{aligned}
 lo + \frac{hi-lo}{2} &= 2m + 1 + \frac{2n+1-(2m+1)}{2} \\
 &= 2m + 1 + \frac{2n-2m}{2} \\
 &= 2m + 1 + (n - m)\frac{2}{2} \\
 &= 2m + 1 + (n - m) + 0 \\
 &= m + n + 1 \\
 &= \frac{2m+2n+2}{2} \\
 &= \frac{2m+1+2n+1}{2} \\
 &= \frac{lo+hi}{2}
 \end{aligned}$$

Solution of exercise 7 Since the only changes involve the body of the loop, the initialization proofs (INIT) remain unchanged with respect to what was examined in the lecture.

Because lo and hi remain unchanged, the proofs of preservation (PRES) become trivial as the conclusion is identical to the conclusion.

The proof of correctness would fail when we consider termination: since lo and hi never change, and the loop guard compares lo and hi , the loop will never terminate whenever $lo \neq hi$ initially, i.e., whenever $n \neq 0$.